UMBRAL

DIAGRAMA DE CLASES Y ARQUITECTURA DEL PROYECTO

Documento técnico basado en los scripts GDScript y en la escena principal de la versión UMBRAL_Menu_Victoria_10_Niveles (última versión con menú de victoria).



Figura 1. Vista general de los once scripts del proyecto y su coordinación desde Main.

11 scripts GDScript documentados. Los nombres UML identifican archivos o nodos: el proyecto no utiliza declaraciones class_name.

01 / UML

Diagrama de clases · Lógica del juego

Modelo de control de partida, configuración, escenario, movimiento y acertijos.

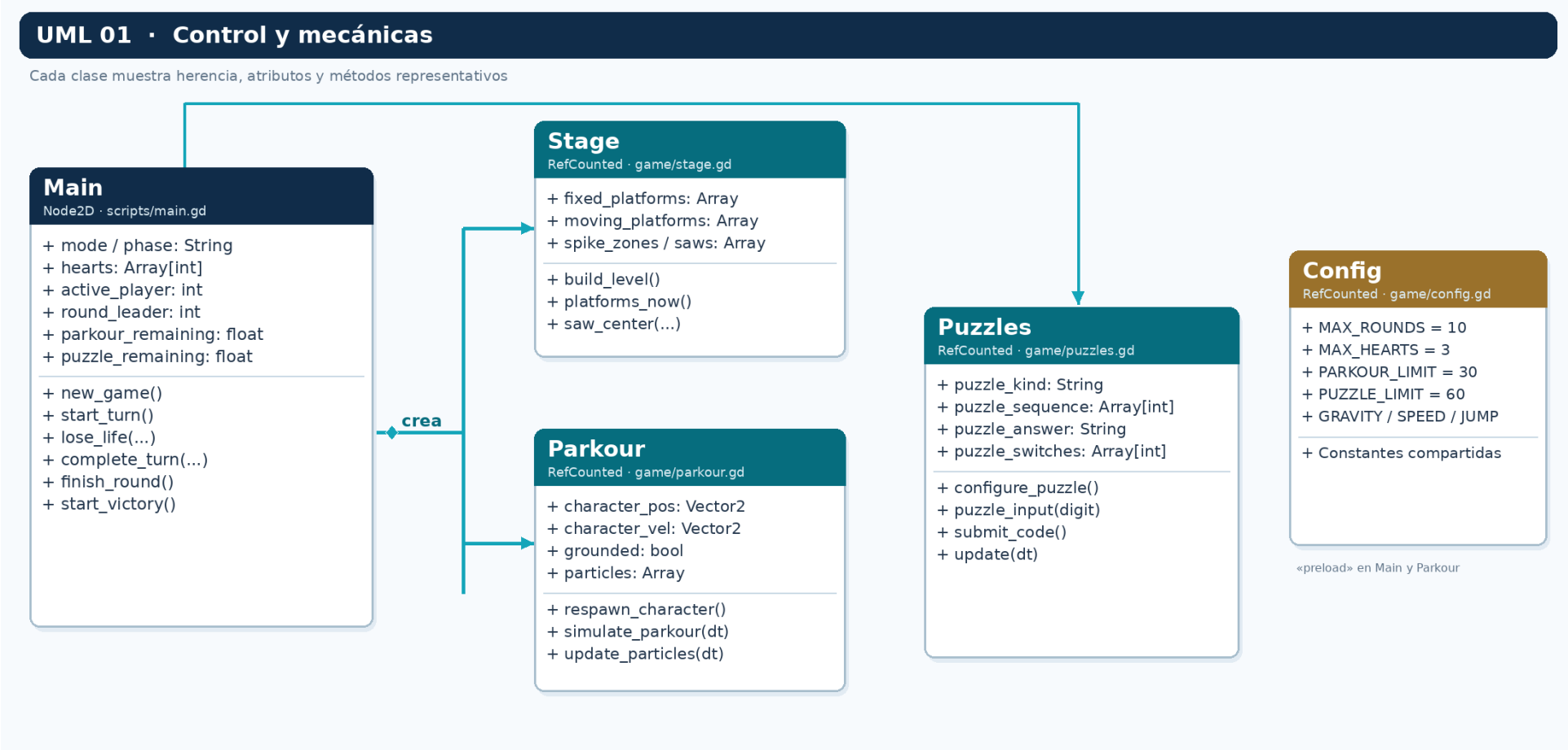


Figura 2. Diagrama UML del núcleo. Cada bloque muestra herencia, atributos seleccionados y operaciones principales.

Main es el orquestador: crea Stage, Parkour y Puzzles con el propio Main como referencia (Stage.new(self), Parkour.new(self), Puzzles.new(self)). Parkour usa Stage para consultar plataformas y riesgos. Config es una dependencia de constantes, no un objeto instanciado.

02 / UML

Diagrama de clases · Presentación y audio

Nodos de escena responsables de interfaz, sonidos y transiciones visuales.

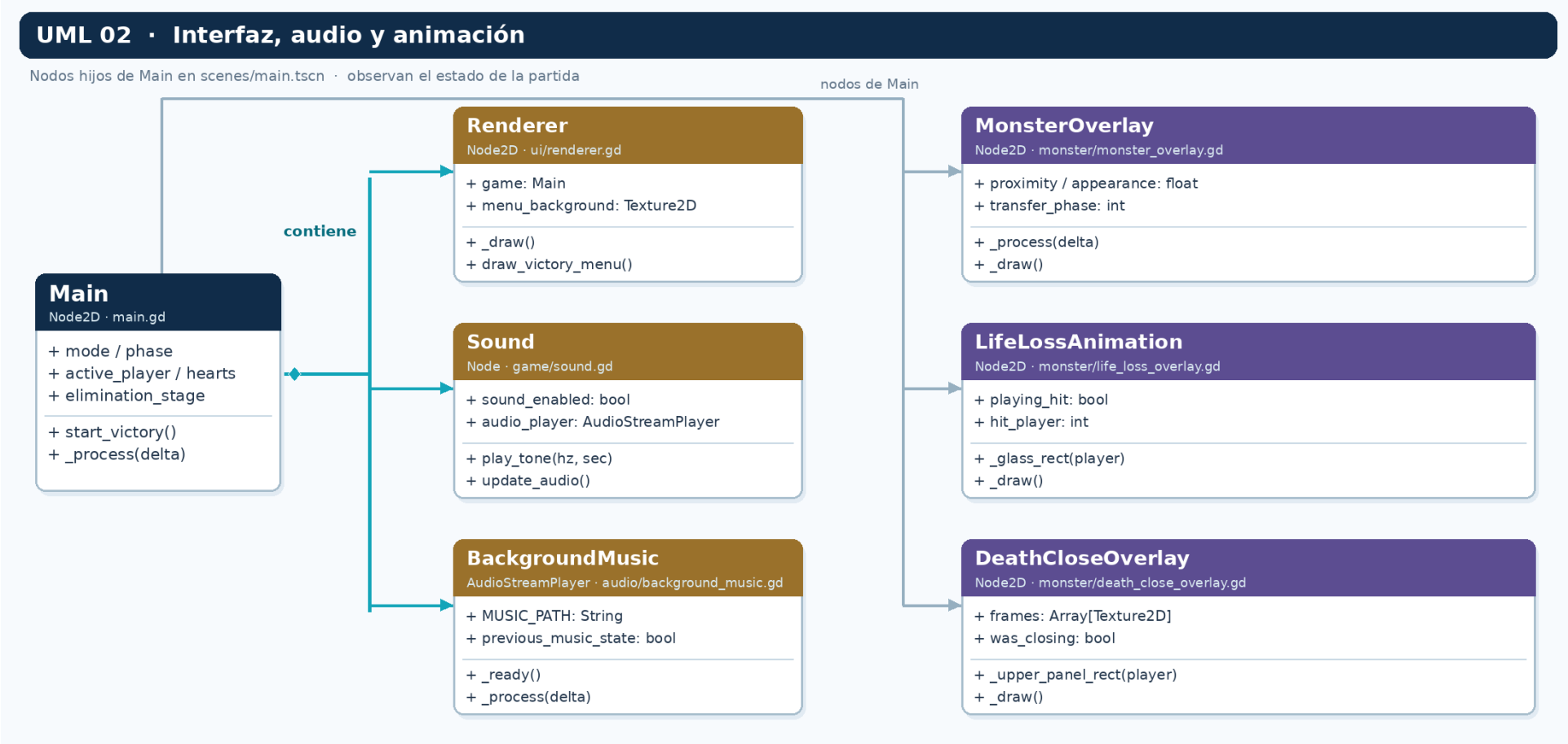


Figura 3. Componentes visuales y de audio. Los nodos son hijos de Main en main.tscn.

Renderer dibuja la interfaz; MonsterOverlay, LifeLossAnimation y DeathCloseOverlay gestionan el monstruo y sus transiciones; Sound crea tonos y BackgroundMusic reproduce BackSong.wav solo durante el parkour.

03 / REFERENCIA

Inventario de clases y responsabilidades

Correspondencia entre cada bloque del diagrama, su script y su función en el juego.

Clase UML	Archivo fuente	Hereda de	Responsabilidad
Main	scripts/main.gd	Node2D	Control de estados, rondas, vidas, turnos y victoria
Config	scripts/game/config.gd	RefCounted	Constantes, tiempos, física y nombres de rondas
Stage	scripts/game/stage.gd	RefCounted	Plataformas, trampas y geometría de niveles
Parkour	scripts/game/parkour.gd	RefCounted	Movimiento, colisiones, caída, meta y partículas
Puzzles	scripts/game/puzzles.gd	RefCounted	Secuencias, códigos y circuitos; variantes por jugador
Renderer	scripts/ui/renderer.gd	Node2D	Dibujo de escenas, HUD, menús y acertijos
Sound	scripts/game/sound.gd	Node	Tonos generados para efectos e interacciones
BackgroundMusic	scripts/audio/background_music.gd	AudioStreamPlayer	Música únicamente durante parkour
MonsterOverlay	scripts/monster/monster_overlay.gd	Node2D	Animación normal, acercamiento y salida del monstruo
LifeLossAnimation	scripts/monster/life_loss_overlay.gd	Node2D	Mordisco activado al reducirse las vidas
DeathCloseOverlay	scripts/monster/death_close_overlay.gd	Node2D	Cierre de tentáculos tras la eliminación

Interpretación UML

Atributos y métodos mostrados en las figuras son los principales, no un volcado exhaustivo. El prefijo “+” indica miembros accesibles del script en GDScript; el código no establece explícitamente niveles de acceso equivalentes a public/private de otros lenguajes.

La composición señala propiedad lógica o pertenencia a la escena, y las flechas discontinuas representan dependencias/consultas. “extends” identifica herencia directa de tipos del motor Godot.

04 / DINÁMICA

Flujos principales y reglas de negocio

Resumen operativo vinculado a las clases del diagrama.

Inicio de partida Main.new_game() → Stage.build_level() → Main.start_turn() → Parkour.respawn_character()

Fase de parkour Main._process() descuenta el tiempo → Parkour.simulate_parkour() → detección de meta o pérdida de vida

Acertijo Main.reach_parkour_finish() → Puzzles.configure_puzzle() → Puzzles.puzzle_input() → Main.complete_puzzle()

Cambio de jugador Main.complete_turn() → siguiente jugador, o Main.finish_round() cuando ambos ya jugaron o el rival fue eliminado

Eliminación Main.lose_life() → LifeLossAnimation (mordisco) → DeathCloseOverlay (tentáculos) → aviso negro; Main reanuda flujo

Victoria Main.finish_round() → Main.start_victory() → MonsterOverlay se retira → Renderer.draw_victory_menu()

Reglas verificadas en el código

- 10 rondas; 3 vidas por jugador al comenzar; 30 segundos base para parkour y 60 segundos para puzzles.
- El líder del turno fija su tiempo de parkour; el retador debe completar el mismo nivel dentro de esa marca.
- El jugador 2 recibe acertijos distintos del jugador 1 en cada ronda.
- Si el nivel final fue superado por al menos un jugador, la victoria se decide por las vidas restantes; empate cuando son iguales.
- Los efectos de sonido dependen de Sound; la música de BackgroundMusic solo se reproduce en modo playing y fase parkour.

Base documental

Rutas comprobadas: project.godot, scenes/main.tscn y los 11 scripts de la carpeta scripts/. El diagrama describe la implementación existente y no introduce clases ni métodos nuevos. Fuente: copia local de UMBRAL_Menu_Victoria_10_Niveles.